

Instituto Tecnológico de Tepic



Ingeniería en Sistemas Computacionales

Programación Web

Actividad 1

Unidad 4

Alumno

- Sandoval López Pedro Alejandro - 23400619

Profesor: Nava Hernández Irving Yair

Tepic, Nayarit, a 13 de Julio del 2026

Investigación Javascript.....	1
Características básicas del lenguaje.....	1
Estructuras de control.....	3
Sentencias condicionales.....	3
Sentencias de bucle.....	5
Declaraciones de salto.....	6
Tipos de datos (primitivos y referencias).....	7
Funciones.....	9
Declaradas.....	9
Expresadas.....	9
Flechas.....	10
Objetos y Arrays.....	11
Objetos (Object).....	11
Arrays (Arreglos).....	11
Eventos y manipulación del DOM.....	11
Eventos.....	11
Manipulación del DOM.....	12
Asincronía.....	12
Callbacks.....	13
Promesas.....	13
Async/Await.....	14
Cuadro Sinóptico.....	15

Investigación Javascript

Javascript es un lenguaje de programación utilizado para crear páginas web dinámicas e interactivas. Es el lenguaje de scripting más utilizado en la web y lo emplean los desarrolladores para añadir funcionalidad a sitios web y aplicaciones.

Puede utilizarse para crear interfaces de usuario interactivas, crear aplicaciones del lado del servidor, conectar bases de datos y mucho más.

Características básicas del lenguaje

- Es un lenguaje multiplataformas

Javascript es un lenguaje que puede ser utilizado con muchos marcos y plataformas, así mismo puede ser ejecutado en cualquier máquina independiente de su sistema operativo.

- Es un lenguaje interpretado y orientado a objetos

Con esto nos referimos a un tipo de lenguaje que permite la definición de tipo de datos y su forma de operar. Es decir, cada objeto debe ser distinguido y esto permite tanto mayor organización como prevención de errores. Al ser interpretado, puede ser ejecutado en el navegador sin tener que ser compilado previamente.

- Es un lenguaje de tipado débil y alto nivel

Javascript es un lenguaje que puede ser fácilmente comprendido debido a su semejanza al lenguaje natural, de ahí que sea considerado de alto nivel. Al mismo tiempo, es de tipado débil no tipado y por tanto, no permite al desarrollador definir el tipo de las variables al declararla.

- Cuenta con una gran cantidad de librerías y frameworks

React, Angular o [Node.js](#) por ejemplo, que facilitan a los desarrolladores crear sus proyectos de una forma más rápida y con menos líneas de código. Además, estas librerías permiten el uso de ciertas tecnologías como AJAX o JSON, entre otras.

- Se puede ejecutar con asincronía

Javascript es un lenguaje asíncrono, lo que significa que se pueden ejecutar múltiples tareas simultáneamente sin afectar al rendimiento de la página.

- Permite trabajar con JSON y conectarse a bases de datos

Javascript puede utilizarse para trabajar con JSON (JavaScript Object Notation), que es un formato ligero de intercambio de datos utilizado para intercambiar y almacenar datos.

Permite crear, analizar y encadenar objetos, por lo que resulta útil para crear aplicaciones web dinámicas.

Además, JavaScript también puede utilizarse para conectarse a bases de datos como MySQL y MongoDB. Esto permite a los desarrolladores crear potentes aplicaciones web que pueden interactuar con grandes cantidades de datos almacenados en estas bases de datos.

Estructuras de control

JavaScript ofrece un amplio conjunto de estructuras de control, categorizadas en tres tipos principales:

Sentencias condicionales

Estas estructuras permiten que el código tome decisiones en función de ciertas condiciones. Ejecutan bloques de código específicos solo cuando se cumplen las condiciones definidas. Las sentencias condicionales esenciales son:

- La if instrucción condicional

Esta es la instrucción condicional más básica. Evalúa una condición y ejecuta un bloque de código si la condición es verdadera. Opcionalmente, se puede incluir un bloque `else` que se ejecuta si la condición es falsa.

```
let grade = 85;
if (grade >= 90) {
  console.log("Excellent!");
} else {
  console.log("Keep practicing!");
}
```

- La else if instrucción

Esto permite encadenar varias condiciones dentro de una instrucción if. Permite comprobar varias cosas una por una y solo ejecuta el código para la primera que sea verdadera.

```
let grade = 85;

if (grade >= 90) {

  console.log("Excellent!");

} else if (grade >= 80) {

  console.log("Great job!");

} else {

  console.log("Keep practicing!");

}
```

- La switch instrucción

Esta estructura se utiliza para ramificaciones múltiples basadas en el valor de una expresión. Proporciona una alternativa más limpia a las instrucciones if anidadas cuando se trata de varias condiciones posibles que se evalúan con valores diferentes.

```
let day = "Tuesday";

switch (day) {

  case "Monday":

    console.log("Start of the week!");

    break;

  case "Tuesday":

  case "Wednesday":

  case "Thursday":

    console.log("Midweek grind!");

    break;

  case "Friday":

    console.log("TGIF!");

}
```

```
break;  
default:  
    console.log("Enjoy the weekend!");  
}
```

- El operador ternario (operador condicional)

Este operador compacto proporciona una forma abreviada de escribir expresiones condicionales sencillas. Evalúa una condición y devuelve uno de dos valores según el resultado.

```
let message = age >= 18 ? "You are eligible to vote." : "Not  
eligible to vote.";
```

Sentencias de bucle

Las sentencias de bucle son fundamentales para iterar a través de colecciones de datos, ejecutar bloques de código repetidamente hasta que se cumpla una condición y automatizar tareas repetitivas. JavaScript ofrece tres estructuras de bucle principales:

- El for bucle

Este práctico bucle te permite repetir una acción una y otra vez fácilmente. Lleva un registro de tu progreso mediante un contador y se detiene cuando ya no se cumple una condición.

```
for (let i = 0; i < 5; i++) {  
    console.log("Iteration:", i);  
}
```

- El while bucle

Este bucle continúa ejecutando un bloque de código mientras una condición específica se mantenga verdadera. La condición se verifica antes de cada iteración, y el bucle continúa mientras la condición se evalúe como verdadera.

```
let count = 0;
while (count < 3) {
  console.log("Count:", count);
  count++;
}
```

- El do-while bucle

Este bucle garantiza que el bloque de código se ejecute al menos una vez, incluso si la condición inicial resulta falsa. La condición se comprueba después de que se ejecute el bloque de código. El bucle continúa iterando mientras la condición siga siendo verdadera.

```
let input = "";
do {
  input = prompt("Enter your name (or 'quit' to exit):");
} while (input.toLowerCase() !== "quit");
console.log("Hello,", input);
```

Declaraciones de salto

JavaScript proporciona dos instrucciones de salto principales que cambian el flujo normal de ejecución dentro de los bucles:

- La break instrucción

Esta instrucción indica que se debe salir del bucle, incluso si este aún no ha finalizado. Se suele utilizar para salir de un bucle antes de que este se detenga cuando se cumple una condición específica dentro del mismo.

```
let numbers = [1, 5, 2, 8, 3];
```

```
for (let i = 0; i < numbers.length; i++) {
  if (numbers[i] % 2 === 0) {
    console.log("Found an even number:", numbers[i]);
    break;
  }
}
```

- La continue instrucción

Esta instrucción omite la iteración actual del bucle que la contiene y pasa a la siguiente. Permite omitir iteraciones específicas de forma selectiva, según las condiciones dentro del bucle.

```
let numbers = [1, 5, 2, 8, 3];
for (let i = 0; i < numbers.length; i++) {
  if (numbers[i] % 2 === 0) {
    continue;
  }
  console.log("Odd number:", numbers[i]);
}
```

Tipos de datos (primitivos y referencias)

Un tipo de datos sirve para definir el tipo y el comportamiento de los datos: En otras palabras, con esto podemos decirle al compilador o al intérprete cómo pretendemos usar una parte de los datos. La mayoría de los lenguajes de programación admiten tipos de datos básicos como números, booleanos, cadenas, etc.

En JavaScript podemos decir que admite ocho tipos de datos: número, BigInt, booleano, string o cadena, valor nulo, indefinido, símbolo y objeto.

- Número (number): El tipo number representa tanto números enteros como de punto flotante.
- BigInt: En JavaScript, el tipo “number” no puede representar valores enteros mayores que $(2^{53}-1)$ (eso es 9007199254740991), o menor que $-(2^{53}-1)$ para negativos. Es una limitación técnica causada por su representación interna. Para la mayoría de los propósitos es suficiente, pero a veces necesitamos números realmente grandes. Por ejemplo, para criptografía o marcas de tiempo de precisión de microsegundos.
- String: Un string en JavaScript es una cadena de caracteres y debe colocarse entre comillas.
- Boolean (tipo lógico): El tipo booleano tiene sólo dos valores posibles: true y false.
- El valor “null” (nulo): El valor especial null no pertenece a ninguno de los tipos descritos anteriormente.
- El valor “undefined” (indefinido): El valor especial undefined también se distingue. Hace un tipo propio, igual que null. El significado de undefined es “valor no asignado”.
- Object y tipos de referencia: El tipo object es especial. Todos los demás tipos (number, string, boolean, bigint, undefined, null y symbol) se consideran

primitivos, ya que almacenan un único valor directamente. En cambio, los objetos son tipos de referencia, porque la variable no almacena el objeto directamente, sino una referencia a su ubicación en memoria. Dentro de los tipos de referencia se encuentran Object, Array, Function, Date, Map, Set y otros objetos especiales.

Funciones

Declaradas

Probablemente, la forma más común de crear las funciones, de esta manera podemos ejecutar la función saludar() incluso antes de haberla creado, ya que este tipo de función es compatible con el hoisting, que es una característica de JavaScript que, en resumen, primero busca las declaraciones de funciones y luego procesa el código.

```
1. saludar(); // Hola
2.
3. function saludar(){
4.   return "Hola";
5. }
```

Expresadas

Esta forma permite guardar una función dentro de una variable, para luego ejecutar dicha variable. Se trata de un enfoque diferente al método anterior pero fundamentalmente hace lo mismo con algunas diferencias.

```
1. const saludo = function saludar() {
2.   return "Hola";
3. };
4.
5. saludo(); // 'Hola'
```

Con este nuevo enfoque, estamos creando una función en el interior de una variable, lo que nos permitirá posteriormente ejecutar la variable (como si fuera una función). Se observa que el nombre de la función “saludar” pasa a ser inútil, ya que si intentamos ejecutar saludar() nos dirá que no existe y si intentamos ejecutar saludo() funciona correctamente.

La diferencia fundamental entre las funciones por declaración y las funciones por expresión es que estas últimas sólo están disponibles a partir de la inicialización de la variable. Si “ejecutamos la variable” antes de declararla, nos dará un error.

Flechas

Las Funciones Flechas son una forma corta de escribir funciones que aparece en JavaScript a partir de ECMAScript 6. Básicamente, se trata eliminar la palabra `function` y añadir `=>` antes de abrir las llaves:

```
1. const func = function () {  
2.   return "Función tradicional";  
3. };  
4.  
5. const func = () => {  
6.   return "Función flecha";  
7. };
```

Sin embargo, las funciones flecha tienen algunas ventajas a la hora de simplificar código bastante interesante:

- Si el cuerpo de la función solo tiene una línea, podemos omitir las llaves `{}`
- Además, en ese caso, automáticamente se hace un `return` de esa única línea, por lo que podemos omitir también el `return`.
- En el caso de que la función no tenga parámetros, se indica como en el ejemplo anterior: `() =>`
- En el caso de que la función tenga un solo parámetro, se puede indicar simplemente con el nombre del mismo: `e =>`
- En el caso de que la función tenga 2 o más parámetros, se indican entre paréntesis: `(a, b) =>`

Por lo tanto, el ejemplo anterior se puede simplificar aún más:

```
1. const func = () => "Función flecha."; // 0 parámetros: Devuelve  
   "Función flecha"  
2. const func = (e) => e + 1; // 1 parámetro: Devuelve el valor de e + 1
```

```
3. const func = (a, b) => a + b; // 2 parámetros: Devuelve el valor de a + b
```

Las funciones flecha hacen que el código sea mucho más legible y claro de escribir, mejorando la productividad y la claridad a la hora de escribir código.

Objetos y Arrays

Objetos (Object)

Un objeto es una estructura que permite almacenar información mediante pares clave: valor. Se utiliza para representar entidades con varias propiedades y métodos.

```
const persona = {  
  nombre: "Pedro",  
  edad: 22,  
  estudiante: true  
};  
  
console.log(persona.nombre);  
console.log(persona.edad);
```

En este ejemplo, el objeto persona tiene tres propiedades: nombre, edad y estudiante.

Arrays (Arreglos)

Un array es una estructura que permite almacenar varios elementos en una sola variable. Cada elemento ocupa una posición llamada índice, comenzando desde el 0.

Ejemplo:

```
const colores = ["Rojo", "Azul", "Verde"];  
  
console.log(colores[0]); // Rojo  
console.log(colores[2]); // Verde
```

Eventos y manipulación del DOM

Eventos

Los eventos son acciones que ocurren en una página web y que pueden ser detectadas por JavaScript para ejecutar una función. Estas acciones pueden ser realizadas por el usuario, como hacer clic en un botón, escribir en un campo de texto o mover el mouse.

```
const boton = document.getElementById("btnSaludar");

boton.addEventListener("click", function() {
    alert("¡Hola!");
});
```

Manipulación del DOM

El DOM (Document Object Model) es la representación de una página HTML en forma de árbol de objetos. JavaScript puede acceder a esos objetos para modificar el contenido, los estilos o la estructura de la página.

Algunas acciones comunes son:

- Seleccionar elementos HTML.
- Cambiar el contenido de un elemento.
- Modificar estilos CSS.
- Agregar o eliminar elementos.
- Responder a eventos del usuario.

Ejemplo:

```
const titulo = document.getElementById("titulo");

titulo.textContent = "Bienvenido";
titulo.style.color = "blue";
```

Asincronía

La asincronía es una característica de JavaScript que permite ejecutar tareas que pueden tardar tiempo, como consultar una API o leer un archivo, sin detener la ejecución del

resto del programa. Esto permite que la aplicación siga respondiendo mientras espera el resultado de esas operaciones.

Callbacks

Un callback es una función que se pasa como argumento a otra función para que sea ejecutada cuando una tarea haya finalizado.

Ejemplo:

```
function saludar(nombre, callback) {  
    console.log("Hola " + nombre);  
    callback();  
}  
  
saludar("Pedro", function() {  
    console.log("Bienvenido.");  
});
```

Promesas

Una promesa es un objeto que representa el resultado futuro de una operación asíncrona. Una promesa puede tener tres estados:

- Pendiente (Pending): la operación aún no termina.
- Cumplida (Fulfilled): la operación se realizó correctamente.
- Rechazada (Rejected): ocurrió un error durante la operación.

Ejemplo:

```
const promesa = new Promise((resolve, reject) => {  
    resolve("Operación completada");  
});  
  
promesa.then(resultado => {  
    console.log(resultado);  
});
```

Async/Await

async y await son palabras clave que permiten escribir código asíncrono de forma más clara y fácil de leer, haciendo que se parezca al código secuencial.

- async indica que una función es asíncrona.
- await espera el resultado de una promesa antes de continuar con la ejecución.

Ejemplo:

```
async function obtenerMensaje() {  
    const mensaje=await Promise.resolve("Hola");  
    console.log(mensaje);  
}  
  
obtenerMensaje();
```

Cuadro Sinóptico

CUADRO SINÓPTICO PROG. WEB

PEDRO ALEJANDRO SANDOVAL LOPEZ

